

THEOREMFORCES

再現可能な Lean 証明のための 公開 certificate layer

A Public Certificate Layer for Reproducible Lean Proofs

Status Audited design specification

Version 0.3.0

Date 2026-08-09

本書は、TheoremForces の目的、trust model（信頼モデル）、certificate 仕様（証明書仕様）、judge architecture（判定基盤）、[semantic provenance](#)（意味論的来歴）、security（安全性）、governance（運営統治）、および 2026-2027 年の実行計画を示す。実装済み機能と目標状態を明確に区別し、[Lean acceptance](#)（Lean による機械的受理）を数学的 [endorsement](#)（数学的承認）と混同しないことを設計の中心に置く。

0.3.0 security revision: adversarial audit で発見した certificate hash の自己参照、append-only log の不備、runner 単独信頼、artifact 再生成、未確定 canonicalization、retention と governance の矛盾を解消する normative correction を追加した。Gate C 完了までは public L1 issuance を停止する。

TheoremForces Project

<https://theoremforces.org/>

Independent project. Not affiliated with, endorsed by, or sponsored by Lean FRO, the Lean project, or the mathlib maintainers.

要旨

TheoremForces は、人間と AI が作る Lean 4 の conjecture と proof body を、固定された Lean / Mathlib 環境で検証し、その結果を再現・引用・再利用可能な certificate として記録するための独立 platform である。中心的な問いは「誰が一番速く解いたか」だけではない。何が、どの環境で、どの axiom と依存関係のもとに検証され、第三者がどこまで再現できるかである。

本書は certificate を単一の badge として扱わず、(1) kernel acceptance、(2) artifact provenance、(3) active environment での reverification、(4) informal intent review、(5) curation の五層に分ける。公開 beta の現状には、外部利用、review quorum、artifact reconstruction、sandbox enforcement、toolchain migration に未解決課題がある。したがって今後の優先順位は、機能拡張や課金ではなく、trust reset、research lighthouse、external adoption、review network、条件付き monetization の順とする。

Abstract

TheoremForces is an independent platform for checking Lean 4 conjectures and proof bodies in a pinned Lean / Mathlib environment and preserving the results as reproducible, citable, and reusable certificates. The central question is not only who solved a problem, but exactly what was checked, under which environment and axiom policy, with which certificate dependencies, and to what extent an independent party can reproduce the result.

This white paper separates five assurance layers: kernel acceptance, artifact provenance, reverification under the active environment, review of alignment with informal intent, and mathematical curation. It documents both the public-beta system and a proposed target architecture. The execution strategy is deliberately gated: repair trust-critical gaps first, demonstrate value through real research collections second, grow a small retained external community third, and open paid pilots only after reliability and operational thresholds are met.

文書の読み方

- **Current:** 2026-08-09 時点で公開 UI、公開 API、公開 repository から確認できる状態。
- **Target:** 本書が提案する certificate schema v2 と production architecture。
- **Gate:** 次の phase へ進むための measurable exit condition。
- 本版は公開文書と公開実装に対する adversarial audit を反映するが、継続的な外部 security review の代替ではない。

目次

1	Version 0.3 normative security correction	1
1.1	Normative scope and conformance	1

1.2	Non-self-referential certificate objects	1
1.3	Acceptance, exact artifacts, and independent verification	2
1.4	Artifact and environment identity	3
1.5	Axioms and certificate dependencies	4
1.6	Append-only log and registry semantics	4
1.7	Sandbox, privacy, abuse, and operations	5
1.8	Release gates and SLO	5
2	問題設定	5
2.1	証明が生成される時代の provenance gap	6
2.2	対象ユーザー	6
2.3	非目標	7
3	設計原則	7
4	Assurance model	8
4.1	五つの独立した assurance layer	8
4.2	Acceptance predicate	8
4.3	Axiom policy	9
5	System architecture	10
5.1	Current public description	10
5.2	Target pipeline	10
6	Certificate schema v2	13
6.1	Design goals	13
6.2	Canonicalization	14
6.3	Lifecycle and supersession	15
6.4	Independent verifier	15
7	Semantic provenance and review	15
7.1	なぜ kernel check と意味 review を分けるのか	15
7.2	Review dimensions	16
7.3	AI disclosure	16
8	Security and abuse resistance	16
8.1	Threat model	16

8.2	Fail-closed resource enforcement	17
8.3	Secret handling and incident response	18
9	Public beta snapshot and trust reset	18
9.1	Snapshot: 2026-08-07	18
9.2	Observed trust gaps	18
9.3	Trust reset plan	19
10	Research workflow and interoperability	19
10.1	Flagship collection	19
10.2	Temporal claim consistency pilot	20
10.3	Relationship to mathlib	21
10.4	Academic citation	21
10.5	Preservation tiers	21
11	Governance and sustainable operation	21
11.1	Small-team operating model	21
11.2	Change governance	22
11.3	Business boundary	22
12	Roadmap: 2026-08 to 2027-07	23
12.1	Phase 0: first 48 hours	23
12.2	Phase 1: trustworthy beta	23
12.3	Phase 2: research lighthouse	24
12.4	Phase 3: review and retention	24
12.5	Phase 4: paid pilot decision	24
12.6	Phase 5: evidence-driven scale	24
13	Metrics and decision gates	24
13.1	North Star	24
13.2	Dashboard	25
13.3	Feature decision rule	25
14	Research questions outside the L1 acceptance meaning	25
15	Conclusion	26

付録 A	Certificate v2 envelope example	26
付録 B	Verification algorithm	28
付録 C	Operational SLO proposal	29
付録 D	Glossary (用語集)	29
References		30

1 Version 0.3 normative security correction

Production status: L1 issuance is disabled

Version 0.2 の certificate hash、transparency receipt、runner trust boundary には、仕様どおりに安全な実装を作れない production blocker があった。本節は本書の他の節と衝突する場合に優先する normative correction である。公開 site の閲覧機能と historical record は運用できるが、Gate 0-C が完了するまで新規 record を **L1 accepted**、**verified**、**fully reproducible**、または **append-only witnessed** として発行してはならない。Historical v1 record は legacy-incomplete と表示し、L1 v2 と同等に数えない。

1.1 Normative scope and conformance

本節、L1 companion specification の normative requirement catalog、Gate 0-C、および repository に置く versioned JSON Schema / golden vectors / adversarial fixtures を normative とする。実装は requirement ID ごとに automated conformance test と evidence URI を持たなければならない。未解決の semantic decision、test evidence の欠落、または checker divergence が一件でもあれば issuance flag は false のままとする。

Requirement	Normative rule
TF-CONF-001	各 MUST に stable ID、owner、test、release evidence を対応させ、unknown / skipped test を pass と数えない。
TF-CLAIM-001	Exact blob、external log receipt、full-chain verifier が揃うまで再現性・追記性を断定しない。UI/API/CLI は assurance layer と不足 evidence を別々に示す。
TF-ATOM-001	Certificate issuance、accepted transition、economy / reputation credit、ledger event、bounty settlement の multi-row mutation は一つの atomic persistence boundary で commit する。現行 Prisma D1 adapter 経路を transaction と見なさず、atomic design と crash / concurrency test が合格するまで全経路を fail-closed にする。
TF-OPS-001	False acceptance、artifact mismatch、checker divergence、log fork、signing key incident の一件で 5 分以内に automatic mint pause する。
TF-GOV-001	Trust-critical release は独立した二人の approver、protected main、signed release、rollback plan を要求する。第二 approver がいなければ mint を停止する。

1.2 Non-self-referential certificate objects

公開 certificate は単一 JSON object ではなく、次の三つの独立 object からなる。Receipt と signature を core の hash 対象へ戻してはならない。

1. **CertificateCore**: statement、exact artifacts、logical environment、target、axiom / certified dependencies、resource outcome、policy checkpoint を含む immutable payload。
2. **IssuanceAttestation**: core digest、issuer、verifier quorum、job expectation、result attestation、issuance policy を拘束する signed envelope。Signatures 自身は signed payload の外側に置く。
3. **LogReceipt**: core digest と issuance digest を leaf にした transparency inclusion / consistency evidence。後から witness を追加しても core digest と issuance digest は変わらない。

RFC 8785 JCS と I-JSON 制約を使用し、duplicate key、lone surrogate、non-finite number、negative zero、schema が許可しない null / unknown field を拒否する。文字列を NFC 等へ暗黙変換してはならない。

$$d_C = \text{SHA256}(\text{UTF8}(\text{TF-CERT-CORE-V2}) \parallel 00_{16} \parallel \text{JCS}(\text{CertificateCore})), \quad (1)$$

$$d_I = \text{SHA256}(\text{UTF8}(\text{TF-ISSUANCE-V1}) \parallel 00_{16} \parallel \text{JCS}(\text{IssuancePayload})), \quad (2)$$

$$m_I = \text{UTF8}(\text{theoremforces.l1.signature.v2}) \parallel 00_{16} \parallel \text{JCS}(\text{IssuancePayload}). \quad (3)$$

Digest 表記は lowercase sha256: と 64 lowercase hexadecimal、署名は Ed25519 / RFC 8032、base64url without padding とする。Issuer、builder、verifier、log は別 key ID と validity interval を使い、発行時 key checkpoint、rotation overlap、revocation reason、compromise handling を公開する [23, 24]。

TF-CERT-001 - cycle-free identity

LogReceipt の追加・更新は d_C と d_I を変えてはならない。Ledger module 名は final certificate hash から導出せず、発行前に確定する opaque artifact ID の 128 bit 以上の domain-separated digest から導出する。Module source は未確定の certificate hash を埋め込まない。Core の任意の 1 byte の変更は d_C を変える。少なくとも二言語の golden vector が一致しなければ release しない。

1.3 Acceptance, exact artifacts, and independent verification

Web / API は dispatch 前に immutable **JobExpectation** を保存する。最低でも attempt ID、job nonce、challenge artifact digest、raw upload digest、entrypoint digest、canonical manifest digest、bundle root、generated source digest、resolved dependency manifest digest、logical environment digest、policy digest、target identity を含む。Runner は同じ値、run ID、attempt number、pre/post input root、resource outcome を署名した **ResultAttestation** として返す。別 attempt の callback、重複 nonce、late callback、値の相違を reject する。

受理 state machine は received -> dispatched -> result-attested -> independently-verified -> blobs-durable -> log-integrated -> published の単調遷移とする。Public certificate、accepted status、reputation credit は最後の遷移を一度だけ atomic に commit した後にのみ可視化する。Object store、database、queue、log を架空の単一 transaction と見なさず、transactional outbox と idempotency key を使う。Retry は新しい attempt であり、元 attempt を上書きしない。

Atomic Persistence Gate (Gate C mandatory precondition)

2026-08-09 時点の Prisma D1 adapter は、この受理境界に必要な multi-statement atomic transaction を提供しない。したがって certificate issuance、accepted status、economy / reputation credit、ledger event、bounty settlement のうち一つでも複数 row にまたがる mutation は、個別の create / update 呼出しの列を transaction と扱ってはならず、public 経路を fail-closed に保つ。解除には、(a) D1 batch() と transactional outbox に対する invariant の証明、(b) Durable Object による single-writer serialization、または (c) multi-statement transaction を提供する database のいずれかを ADR で固定し、各 statement 直前・直後の crash、duplicate / late callback、retry、二重 submit、並行 finalize、out-of-order delivery を網羅した fault-injection / concurrency test で、partial certificate、二重 credit、orphan ledger event、二重 bounty payment が 0 件であることを示さなければならない。External object / queue / transparency log は同じ DB transaction の一部と偽らず、durable outbox と idempotent consumer で接続する。この gate の release evidence がない限り Gate C の 30 日観測を開始しない。

TF-ACC-001 - conjunctive, fail-closed acceptance

published は exact artifact match、immutable expectation match、one target、trusted challenge binding、Comparator equality、sandboxed builder completion、sandbox 外の proof export validation、Lean checker PASS、independent checker PASS、structured transitive axiom policy PASS、dependency checkpoint PASS、resource policy PASS、durable blob availability、issuer signature、log integration の conjunction だけで成立する。Reject、crash、timeout、unsupported、parse error、missing evidence は accepted ではなく rejected または indeterminate である。

User-controlled Lean source、macro、command、tactic、plugin、.olean は hostile input とする。通常の Lean process 成功は builder evidence にすぎない。debug.skipKernelTC や unchecked declaration injection を防ぐため、trusted Challenge と Comparator で statement を拘束し、exported proof を別 trust domain の Lean checker と独立実装 checker で再検査する。Builder callback 単独で issuer signer を起動してはならない [16, 25]。

1.4 Artifact and environment identity

artifact.upload_raw_sha256、artifact.entrypoint_raw_sha256、artifact.manifest_sha256、artifact.bundle_root_sha256 は異なる意味を持つ必須 field とする。Bundle root は "theoremforces.artifact.v1" || 0x00 || JCS(canonicalManifest) の SHA-256 とする。Manifest は path 昇順、regular-file raw bytes、size、fixed mode を commit する。Absolute path、empty / dot / dot-dot segment、backslash、NUL、case-fold collision、symlink、hardlink、device、FIFO、socket を reject する。Archive は compressed bytes を先に hash / size 検査し、entry 数、expanded size、depth、ratio を制限する。

Transport bytes、decoded UTF-8 bytes、generated source bytes は別 blob として insert-only CAS に保存する。Generator executable / image digest と exact source map を記録し、後日 template から見かけを再生成して judged blob の代替にしてはならない。Source identity のため Unicode を正規

化しない一方、UI は bidi control と default-ignorable を可視化し、escaped byte view と confusable warning を提供する。

Logical environment は Lean / Lake / Mathlib source commit、package source tree / submodule、manifest、import roots、Lean checker、Comparator、exporter、independent checker、generator、verifier の binary digest、command / option、OCI platform / config / rootfs digest、SBOM、signed build provenance を含む。Moving tag、network resolution、unmanifested file、shared mutable cache を禁止する。Exact pin は安全性の証拠ではないため、known-unsound epoch を止める security allowlist / blocklist を別に運用する。

1.5 Axioms and certificate dependencies

Human-oriented `#print axioms text` の regex parse は acceptance authority にしない。Proof export から primitive assumption と actual certified constant の transitive closure を machine-readable に二つの checker で再計算する。Primitive baseline は exact declaration identity、kind、type digest、origin module digest で `propext`、`Classical.choice`、`Quot.sound` のみを policy version に明記する。`sorryAx`、`compiler trust`、`native computation` 由来 assumption、`custom / unknown axiom` は reject する。

Certificate-backed dependency は無条件の kernel proof ではない。通常 build で stub cache を使う場合も、issuer は全 upstream proof export を topological order で再検査する。UI は primitive assumptions と certified assumptions を同じ trust panel で列挙し、**stub rebuild success** と **full-chain rebuild success** を別状態にする。一つでも certified dependency があれば **axiom-free** または **unconditional proof** と表示しない。

1.6 Append-only log and registry semantics

Transparency tree は RFC 9162 型の domain separation を使用する。**LogLeafV2** は `schema = theoremforces.l1-log-leaf/2`、`core_sha256 = d_C`、`issuance_sha256 = d_I` の三 field だけを持つ object とする。Leaf input は $L = \text{JCS}(\text{LogLeafV2})$ 、leaf は $\text{SHA256}(0x00 \parallel L)$ 、internal node は $\text{SHA256}(0x01 \parallel \text{left} \parallel \text{right})$ とし、odd leaf を複製しない。Signed checkpoint は log ID、tree size、root、timestamp、algorithm、key ID、maximum merge delay を含む。各公開 checkpoint は前 checkpoint からの consistency proof、有効な log signature、少なくとも二つの独立 witness / mirror receipt を持つ。Inclusion proof だけ、daily independent root、mutable Git tag、同一 D1 row は append-only の証拠ではない。MMD 内に統合される前は `pending-log` であり、published certificate と数えない。

Registry は次の二 object に分ける。

- **RegistryEventLog**: issue、status、invalidate、retract、supersede、taint を順序付きで追記する。
- **RegistryStateCheckpoint**: event prefix から決定的に導出する versioned authenticated map。

`validAtIssue(c, R_issue)` と `currentTrustState(c, R_now)` を分離する。後日の invalidation は過去の core を変更せず、全 downstream へ deterministic taint event を追加する。Issuance が参照する checkpoint は issuance より前で、forward reference と cycle を禁止する。Accepted certificate、artifact、attempt / failure、event、checkpoint、receipt は certificate lifetime 中削除または空文字化しない。取

得不能時は `replay-unavailable event` を追記する。

1.7 Sandbox, privacy, abuse, and operations

Hostile build は non-root、no secrets、read-only root / import store、fresh ephemeral workspace、default-deny egress、zero capabilities、seccomp、PID / mount / user namespace または disposable microVM で実行する。Docker / host socket、SSH agent、cloud metadata、credential path を公開しない。CPU、wall time、RSS、PID、open files、disk / inode、output、network、exit code、signal、OOM、timeout、truncation の limit と actual value を signed evidence に含める。Canary secret、network、host file、limit overrun の adversarial corpus を image ごとに通す。

Public mint 前に immutable-publication confirmation、secret / PII scan、quarantine、copyright / illegal content report、takedown tombstone、appeal、legal hold を実装する。Private artifact は unsalted digest を public log に置かず、tenant-scoped encryption、per-object key、cryptographic erasure、dedup / reference count 分離を使う。Stored XSS、log injection、SSRF、path traversal、huge dependency DAG、review Sybil / collusion、reputation farming、plagiarism、doxxing を threat model と test corpus に含め、DAG depth / node cap と moderation audit event を要求する。

1.8 Release gates and SLO

1. **Gate 0 - Spec freeze:** normative schema、JCS / signature / Merkle golden vectors、failure enum、checker quorum、key policy、全 requirement test mapping を確定し、semantic open question を 0 件にする。
2. **Gate A - Adversarial CI:** malicious Lean meta、kernel-skip、statement substitution、custom / native axiom、stale cache、duplicate JSON、Unicode / path collision、archive bomb、callback replay、log fork を全件拒否する。
3. **Gate B - Independent replay:** clean host で対象 sample の 100% を再構成し、Comparator、Lean checker、independent checker、type / value / export / axiom digest が一致する。
4. **Gate C - Production readiness:** 30 日間、必須 evidence 100%、sampled replay 100%、false acceptance 0、mint kill-switch、key revocation、restore、log fork、downstream taint drill を完了する。30 日 clock の開始前に TF-ATOM-001 の atomic persistence design を固定し、crash / retry / callback replay / parallel-finalize test を全件通す。

初期 SLO は minted integrity mismatch 0 件、log integration 99.9% within 10 minutes、certificate / artifact read availability 99.9% monthly、trust-critical RPO 0、rebuildable analytics RPO 24 hours とする。Critical incident は internal page 5 分以内、public status 60 分以内、affected user initial notice 24 時間以内、monthly restore drill とする。各 SLI は denominator、rolling window、exclusion、measurement source を公開する。Incident が起きなかった日数だけを readiness evidence にしない。

2 問題設定

2.1 証明が生成される時代の provenance gap

Lean は dependent type theory に基づく interactive theorem prover であり、elaborator や tactic が生成した proof term は小さな kernel によって検査される [13, 14]。この構造は、tactic や AI が複雑化しても、最終的な formal derivability を kernel で確認できるという強い基盤を持つ。

しかし、kernel が受理したという事実だけでは、研究上必要な問いのすべてには答えない。

- formal statement は、研究者が意図した informal theorem と一致しているか。
- proof はどの Lean / Mathlib revision と import set で検査されたか。
- sorryAx、custom axiom、compiler trust、既存 certificate への依存は何か。
- judge に投入された exact source と、後から UI が再構成する source は同一か。
- toolchain 更新後も通るか。通らない場合、historical validity と current compatibility をどう分けるか。
- AI が生成した artifact を、人間の review や数学的価値判断とどう区別するか。

TheoremForces は、この gap を certificate ledger と semantic provenance によって埋める。目標は「AI が真理を宣言する platform」ではなく、「Lean が何を受理し、その artifact がどのように作られ、何を意味すると人間が判断したかを分離して残す platform」である [1, 7]。

2.2 対象ユーザー

利用者	主要 job	必要な assurance
数学者	informal problem を Lean statement と proof project に分解する	intent alignment、citation、共同作業
Lean formalizer	proof body を検査し、再利用可能な theorem chain を作る	exact environment、imports、axioms
AI theorem prover 開発者	generated proof を外部 judge で評価し、benchmark record を残す	API、rate limit、machine-readable certificate
研究 group / lab	private work を保存し、公開時に選択的に certificate 化する	access control、export、migration
第三者 verifier	platform operator を全面的に信頼せず検算する	content address、manifest、one-command verifier
reviewer / curator	formal statement と数学的意図・価値を評価する	review trail、conflict policy、retraction

2.3 非目標

TheoremForces は mathlib の代替ではない。一般的で再利用可能な数学は、適切な review を経て mathlib に upstream されるべきである。TheoremForces は、その前後で発生する conjecture、attempt、pinned build artifact、AI disclosure、citation metadata を記録する staging / certificate layer である [2, 18]。

また、Accepted は novelty、importance、peer review、publication readiness を意味しない。Seeds は money、token、security、redeemable asset ではなく、platform 内の reputation signal に限定する。marketplace、cash bounty、off-platform payment は scope 外である [2, 8]。

3 設計原則

P1. Lean accepted は必要条件であり、数学的 endorsement ではない

Kernel acceptance、artifact reproduction、intent review、curation を別々の state として表示する。一つの緑 badge に統合しない。

P2. Exact bytes over regenerated appearance

証明書が hash する対象は、後日の template で再生成した表示ではなく、judge が実際に受け取った byte-for-byte source と bundle である。表示も保存 artifact を source of truth とする。

P3. Pinned is not latest

Active pinned environment（現在の固定検証環境）と upstream stable release（上流の最新安定版）は別の概念である。historical certificate は元の epoch で再現し、current compatibility は migration campaign の結果として別記する。

P4. Fail closed at trust boundaries

Hash、dependency、axiom parse、resource limit、webhook identity のどれかを確認できない場合、Accepted や fully reproducible と表示しない。未知を成功へ丸めない。

P5. Public verification must not require operator trust

Certificate JSON、artifact、manifest、Merkle inclusion proof、verifier を公開し、第三者が独立に検算できる経路を用意する。SLSA provenance と in-toto attestation の考え方を参照し、何が、いつ、どの builder と inputs から生成されたかを machine-readable に残す [20, 21]。

P6. Research value before feature surface

North Star は conjecture 総数や AI-generated submission 数ではない。外部利用者によって作られ、別の proof または研究成果から再利用・引用された provenance-complete certificate 数である。

4 Assurance model

4.1 五つの独立した assurance layer

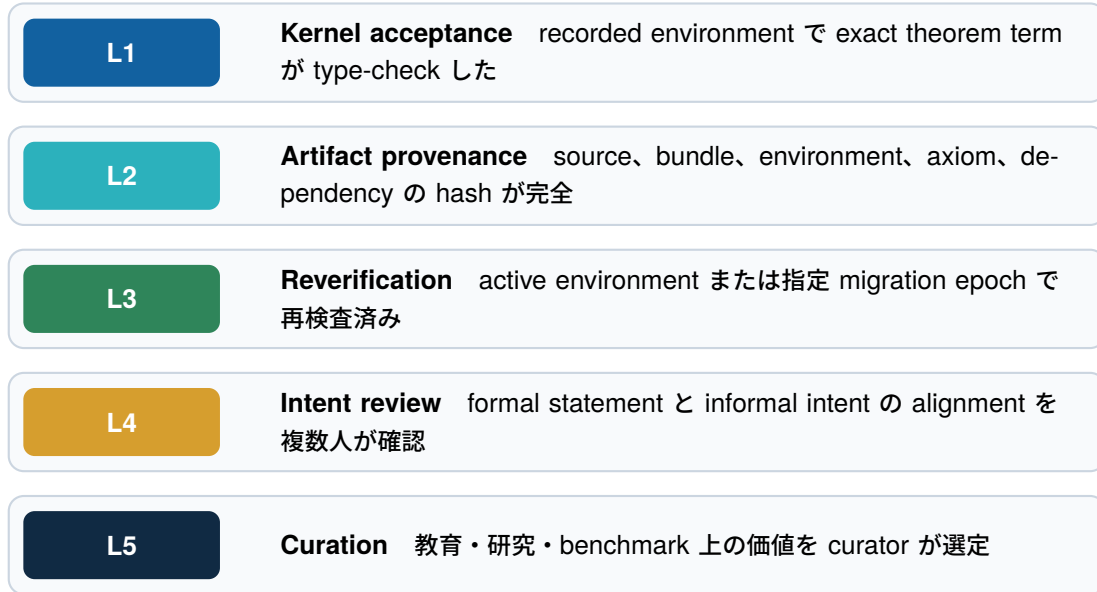


図 1: 一つの badge に畳み込まない assurance stack

L1 は Lean の kernel による derivability の確認である。L2 は、その確認対象と実行環境の provenance を第三者が検算できることを表す。L3 は時間軸を導入し、historical acceptance と current compatibility を分離する。L4 は人間が意図との一致を review した状態、L5 は数学的価値を選定した状態である。

この分離によって、たとえば次の表現が可能になる。

- Lean accepted / artifact mismatch / intent unreviewed
- Lean accepted / provenance complete / needs migration
- Lean accepted / provenance complete / intent aligned / curated
- Lean accepted / provenance complete / intent misaligned / useful accidental lemma

4.2 Acceptance predicate

以下は本節 0.3 の object model を要約する補助式である。Formal statement を s 、proof body を p 、direct certificate imports を I 、generator version を v 、logical environment epoch を E 、primitive axiom whitelist を W 、certificate registry の checkpoint root を R とする。生成 source と judge bundle を

$$A = G_v(s, p, I, E), \quad (4)$$

$$B = \text{Bundle}(A, I, E) \quad (5)$$

と定義する。 $J_E(B)$ を isolated judge の結果、 $P_E(A)$ を primitive axiom set、 $C_E(A)$ を certificate-backed assumptions の transitive set、 $D(B)$ を bundle から計算した direct dependency set、 $N(A)$

を environment delta 上の user target declaration 数、 H を SHA-256 とする。Dispatch 前に ledger が commit した expected digests を h_A^*, h_B^* 、trusted challenge との Comparator 判定を $Q(A)$ 、proof export に対する Lean checker と independent checker の quorum を $V(A)$ とする。各 $c \in C_E(A)$ について、 $\text{ValidCert}(c, E, R)$ は active certificate、statement type hash、同一 logical environment digest、および upstream dependency chain を registry checkpoint R に対して検証する。新規 submission の Accepted 条件は、概念的には次である。

$$\begin{aligned}
 \text{Accept}(s, p, I, E) \iff & J_E(B) = \text{success} \\
 & \wedge N(A) = 1 \\
 & \wedge P_E(A) \subseteq W \wedge \text{sorryAx} \notin P_E(A) \\
 & \wedge \forall c \in C_E(A), \text{ValidCert}(c, E, R) \\
 & \wedge \text{Acyclic}(C_E(A)) \\
 & \wedge D(B) = I \\
 & \wedge H(A) = h_A^* \wedge H(B) = h_B^* \\
 & \wedge Q(A) = \text{pass} \wedge V(A) = \text{pass} \\
 & \wedge \text{LimitsEnforced}(J_E).
 \end{aligned} \tag{6}$$

重要なのは、単なる process exit code 0 ではない点である。単一の実行対象、primitive axiom report、certificate dependency、resource policy、dispatch 前 expectation、independent checker evidence が同一 attestation chain に結び付き、artifact durability と log integration を含む publish transition が一度だけ成立して初めて L1 と L2 が成立する。

4.3 Axiom policy

Lean は definition が直接・間接に依存する axiom を `#print axioms` で列挙する。公式 reference は `sorryAx`、`compiler trust`、`custom axiom` と、標準的な `propext`、`Classical.choice`、`Quot.sound` を区別している [15, 16]。

TheoremForces の policy は次を満たすべきである。

1. `sorryAx` は whitelist に関係なく hard reject。
2. parse failure / missing report は success ではなく infrastructure failure。
3. primitive standard axiom、custom axiom、certificate-backed assumption を UI と schema で別 category にする。
4. certificate dependencies は axiom 名の文字列推測や namespace の一致だけで許可せず、server-resolved bundle manifest、certificate hash、statement type hash、environment digest から確定する。
5. parser version と raw output hash を certificate に含める。

一度 accepted された命題を downstream proof の高速な elaboration cache として使う場合、server は `TheoremForces.Certified.c_<certificate-hash>` のような opaque / axiom stub を生成してよい。ただし、その stub は primitive allowlist W に追加しない。 W は論理原理を管理する安定した policy、certificate registry は既検証 theorem を管理する追記型の object graph であり、trust source が異なる

からである。user が同じ namespace の axiom を自作しても registry record がなければ拒否する。Stub build は certificate-backed assumptions に相対的な成功であり、issuer は署名前に upstream proof export を full-chain replay しなければならない。

二層 versioning で「公理リストの頻繁な更新」を避ける

Lean、Mathlib、package manifest、primitive axiom policy、wrapper rule を immutable な logical epoch E として固定する。accepted certificate は同じ E の append-only registry に追加し、checkpoint root $R_0, R_1, \dots$ だけを更新する。したがって theorem の追加ごとに W や E の version を上げない。import は certificate hash を指し、別 epoch では reverification 後に新しい migrated certificate を発行する。失効時は依存 DAG の downstream を taint する。

5 System architecture

5.1 Current public description

公開 documentation によれば、web application は Next.js on Cloudflare Workers、database は Cloudflare D1、Lean build は外部 isolated judge が担当する。web server 自身は Lean を実行せず、fixed namespace、static filter、type-checking trailer、pinned toolchain、HMAC-authenticated callback を組み合わせる [2, 3, 4]。

公開 judge repository は Lean v4.13.0 と Mathlib v4.13.0 を pin した historical environment と evidence を保持するが、旧 GitHub Actions judge は 2026-08-09 に恒久停止し、stale submission branch を fail-closed で拒否する [10]。この workflow は certificate acceptance authority ではない。v0.3 Gate C の independent-checker pipeline が実装・承認されるまで public L1 issuance は停止し、web、queue consumer、Pi worker、admin fallback の全経路が同じ release gate に従わなければならない。

同日時点の production persistence は Cloudflare D1 と Prisma D1 adapter を使うが、この adapter 経路は certificate、economy / reputation、ledger、bounty の multi-row mutation に必要な atomic transaction boundary を提供しない。したがって current deployment の trust-critical public acceptance / issuance surface は write-disabled であり、公開側には public browsing と historical record の表示だけを提供する。新規 certificate mint、accepted transition、credit、ledger append、bounty settlement、および judge queue delivery は fail-closed に保つ。これは application の全 write を禁止する主張ではない。Private draft 保存、registry の non-issuance maintenance、API の non-issuance write は分離した boundary で継続し得るが、certificate / accepted state / credit / ledger / bounty side effect を生成または変更してはならない。この停止状態は TF-ATOM-001 を満たした証拠ではなく、atomic design の選定と crash / concurrency conformance test 完了まで解除しない。

5.2 Target pipeline

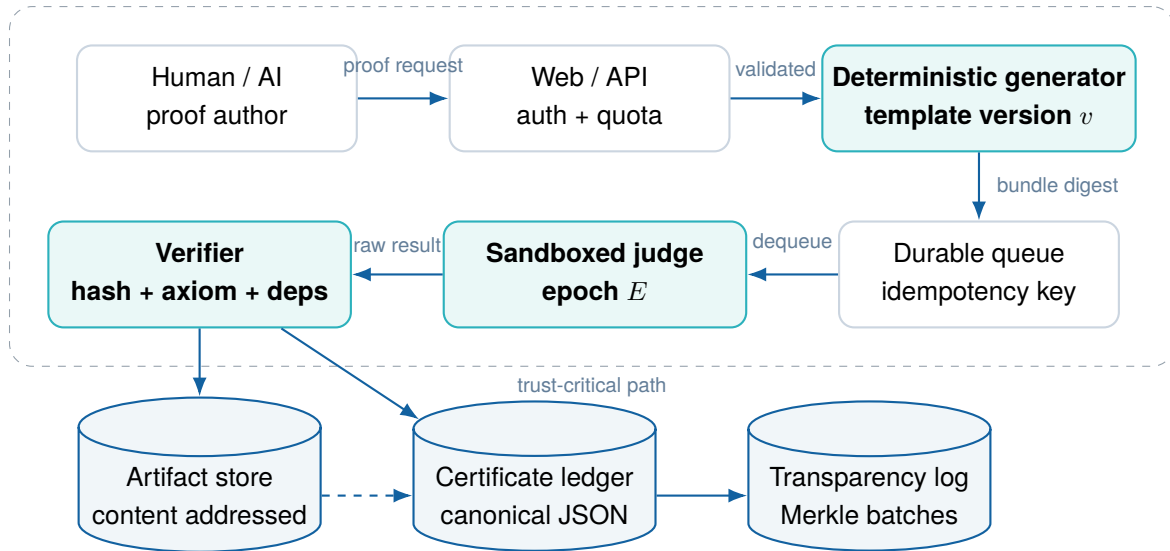


図 2: 提案する end-to-end certificate pipeline

5.2.1 Submission intake

Request は authenticated identity、conjecture id、proof body、AI disclosure、selected certificate imports、client idempotency key を含む。server は quota と content-size cap を先に適用し、static filter の結果を記録する。static filter は defense-in-depth であり、Lean soundness の完全な証明とは扱わない。

5.2.2 Single-target L1 and batch facade

L1 の原子単位は **one attempt = one theorem** とする。user payload に別の top-level theorem / lemma があれば、使われていなくても受理しない。proof 内の have、let、local helper は単一 target の proof term に閉じるため許可できる。共有したい補助定理は、先に別の L1 attempt として certificate 化し、その certificate を依存として参照する。

UI や API は複数定理を一括入力できてよいが、それは L1 の外側にある batch orchestrator が N 個の独立 job に fan-out している facade である。batch 自体を atomic に accepted とせず、各 theorem の certificate と結果を集約する collection manifest を返す。これにより target ambiguity、all-or-nothing failure、resource accounting の混同を避ける。



図 3: 複数定理 UX と単一 target L1 の境界

5.2.3 Deterministic generation

Generator は statement、proof body、server-resolved imports、namespace、type-checking trailer、axiom command を順序を固定して出力する。user payload の top-level target は一つに限定し、wrapper 自身が作る declaration と区別して count する。生成 source は直ちに hash し、template version

とともに immutable artifact store へ保存する。後日の UI は source を再生成せず、この blob を表示する。

Listing 1: 生成 artifact の概念例

```
-- generator: tf-wrapper-v2
-- environment: lean-4.32.2-mathlib-<commit>
-- submission: <id>

import Mathlib
import TheoremForces.Certified.Dep_<certificate_hash>

namespace TheoremForces.Conjectures.<NS>.Submissions.<ID>
-- Exactly one user target declaration is generated.
theorem solution_target : <target type> := by
  <proof body bytes, unchanged>
#print axioms solution_target
end TheoremForces.Conjectures.<NS>.Submissions.<ID>
```

5.2.4 Certificate-backed import registry

accepted theorem の proof artifact は保存し続ける一方、通常の downstream build では毎回その proof body を再生せず、certificate に拘束された opaque constant を使えるようにする。import resolver は次をすべて満たす stub だけを bundle に入れる。

1. certificate が発行時 checkpoint で valid で、現在 status と taint が別に提示され、canonical core、issuance signature、transparency evidence が有効である。
2. stub の exact type hash が certificate の statement type hash と一致する。
3. importing job と dependency の logical_environment_digest が一致する。
4. dependency graph が acyclic で、全 upstream certificate も同じ規則を満たす。

Versioned object	Change cadence	意味
Logical epoch E	rare, reviewed migration	Lean / Mathlib / packages / wrapper / primitive policy
Primitive policy W	rare, governance change	kernel-level assumptions の allow / deny
Registry event prefix R_n	frequent, append-only	issue / status / taint event の 順序付き prefix
Registry state checkpoint	frequent, derived	event prefix から導出した authenticated current state
Certificate hash c	immutable	exact theorem、artifact、environment、dependencies

この仕組みは replay の代替ではなく hot-path cache である。独立 verifier は必要に応じて certificate chain を再帰的に展開し、historical proof artifact を rebuild できる。cross-epoch import は拒否し、

migration replay から新しい certificate と stub を発行する。

5.2.5 Sandboxed judge

Judge image は digest で pin し、起動時 preflight で cgroup v2、network namespace、read-only base filesystem、non-root uid、ephemeral workspace、PID / memory / CPU / wall-clock / output cap を確認する。一つでも enforce できなければ production readiness check を失敗させる。

5.2.6 Verification and minting

Verifier は worker が返した status をそのまま信用しない。job identity、bundle hash、environment digest、axiom report、dependency manifest、resource policy result を照合する。minting と public status update は idempotent transaction とし、callback replay で二重 certificate や二重 reputation credit が発生しないようにする。

5.2.7 Transparency log

Certificate core digest と issuance digest の組を cumulative Merkle log に追加する。LogReceipt は core の外に置き、signed checkpoint、inclusion proof、直前 checkpoint からの consistency proof、少なくとも二つの independent witness receipt を公開する。独立した daily tree、同一 database の root、mutable Git tag、inclusion proof だけでは append-only を主張しない。RFC 9162 の domain separation と consistency algorithm を profile として固定する [22]。

6 Certificate schema v2

6.1 Design goals

v2 schema は「metadata がある」だけでなく、「第三者が同じ object graph を再構成できる」ことを目標とする。SLSA が provenance を artifact の origin、time、builder、inputs に関する verifiable information と定義する考え方を採り入れる [20]。

Object	Required fields	Verification purpose
Statement	canonical text, hash, conjecture id, target count = 1	何を証明したかを一意に固定
Proof body	exact bytes, hash, author disclosure	user input を固定
Generated artifact	source blob digest, module path, template id	judge input の再構成を不要にする
Environment	Lean tag + digest, Mathlib commit, manifest hash, epoch	toolchain を同定
Builder	image digest, architecture, sandbox policy, run id	実行主体と制限を同定
Axioms	primitive names, classification, raw report hash, parser version	primitive trust assumptions を監査
Certified dependencies	cert / type hash, environment digest, registry root, closure hash	graph、same-environment、taint を追跡
Result	exit status, timestamps, log digest, resource outcome	execution result を検算
Issuance attestation	core digest, job expectation, verifier quorum, issuer key / signature	誰がどの evidence から発行したかを認証
Log receipt	log id, tree size, leaf index, inclusion / consistency proof, signed checkpoint, witnesses	append-only history と split view を検算

6.2 Canonicalization

CertificateCore P は RFC 8785 JCS と I-JSON 制約に従う。Duplicate key、lone surrogate、non-finite number、negative zero、schema 外 field を拒否し、array order と null / omission を schema ごとに固定する。Core digest は receipt と signature を一切含まない domain-separated canonical bytes に対して計算する。

$$d_C = \text{SHA256}(\text{UTF8}(\text{TF-CERT-CORE-V2}) \parallel 00_{16} \parallel \text{JCS}(P)). \quad (7)$$

文字列の Unicode normalization や newline 変換を行わない。Direct imports は certificate digest で sort し、Merkle audit path は順序を保持する。Versioned JSON Schema を normative artifact とし、required field、type、hash profile、unknown field、最大 object / array / string size を固定する。Golden vectors を公開し、少なくとも二言語で同じ digest と signature message を生成すること

を test する。

6.3 Lifecycle and supersession

Immutable certificate を後から silent edit しない。問題が発見された場合は状態 record を append する。

- **active**: 現在有効な historical acceptance record。
- **superseded**: 新しい environment / corrected provenance certificate が後継。
- **invalidated**: acceptance predicate または artifact integrity に重大な欠陥。
- **retracted**: informal explanation / authorship / policy 上の理由。kernel record は履歴として残す。
- **legacy-incomplete**: 過去 schema に必要 field がなく、完全再現を主張しない。

Status event 自体にも timestamp、reason code、actor、evidence link、previous event hash を持たせる。

6.4 Independent verifier

Reference CLI は次の UX を目標とする。

Listing 2: 目標とする verification UX

```
theoremforces verify tcl-cert-20260605-0d2nux

[ok] certificate canonical hash
[ok] statement and proof-body hashes
[ok] generated Lean source hash
[ok] judge bundle manifest
[ok] 4 direct certificate dependencies
[ok] dependency type hashes and logical environment
[ok] registry checkpoint and acyclic certificate chain
[ok] Merkle inclusion proof
[ok] sandbox policy attestation
[ok] lake build in recorded environment
[ok] normalized axiom report matches certificate
```

Verification mode は三段階にする。

1. **metadata**: download と hash / Merkle / dependency 検算だけ。
2. **rebuild**: recorded container / toolchain で lake build を再実行。
3. **migration**: selected target epoch で rebuild し、新しい compatibility report を作る。

7 Semantic provenance and review

7.1 なぜ kernel check と意味 review を分けるのか

Lean kernel は formal statement の derivability を確認する。formal statement が informal theorem の翻訳として妥当か、仮定が強すぎないか、定義が意図した対象を表すか、数学的価値があるかは別問

題である。TheoremForces の公開 model は Wild、Registry、Curated の三層を提案している [7]。

Wild

AI-generated conjecture、failed attempt、scaffold、unreviewed certificate を探索として許容する。

Registry

Lean acceptance と machine-verifiable provenance を持つ record。数学的 endorsement ではない。

Curated

intent、axiom safety、essential dependencies、explanation、research / educational value を人間が確認する。

7.2 Review dimensions

Review を一票の like にしない。各 reviewer は少なくとも次を独立に評価する。

1. **Intent alignment:** informal theorem と formal target が一致するか。
2. **Assumption adequacy:** 不要に強い仮定、vacuous premise、empty type 依存がないか。
3. **Definition fidelity:** custom definition が標準的数学概念を正しく表すか。
4. **Dependency safety:** imported certificate の status と semantic role が適切か。
5. **Explanation quality:** 数学者が proof の役割と限界を理解できるか。

Canonical review status は reviewer eligibility、self-review exclusion、conflict of interest、blocking dispute、retraction を考慮する。公開 documentation の 5 人 quorum は high assurance には妥当だが、初期 community では quorum が成立しにくい。そのため UI は 0/5 を単なる未達として見せるだけでなく、各 dimension の partial progress と reviewer shortage を示す。

7.3 AI disclosure

AI 使用は許可し、acceptance を disclosure で差別化しない。ただし、human authorship、AI drafting、repair、autonomous submission の level を machine-readable に記録する。reviewer は「AI を使ったか」ではなく、artifact、intent、dependency、explanation を評価する。

重要なのは、AI disclosure を proof validity の代用品にも、不正検出 detector にも使わないことである。自己申告は provenance の一部であり、kernel acceptance は実際の Lean result、semantic endorsement は review record によって決まる。

8 Security and abuse resistance

8.1 Threat model

Threat	Impact	Required controls
Fake acceptance	証明なしで certificate / reputation を取得	exact job identity、commit / bundle hash、verifier-side result fetch、idempotent mint
Namespace escape	他 submission、statement、judge package を改変	server-owned imports、wrapper namespace、token filter、minimal build context
Elaboration-time code execution	credential / filesystem / network 侵害	no network、non-root、read-only root、seccomp、ephemeral workspace
Resource exhaustion	queue outage、費用増大、他 user の starvation	cgroup CPU / memory / PID、timeout、size cap、fair queue、quota
Axiom bypass	sorryAx や custom axiom を clean と表示	semantic axiom report、fail-closed parser、raw report hash、regression corpus
Artifact substitution	表示 source と judge source が不一致	exact source blob、content address、bundle manifest、independent verifier
Dependency omission	imported theorem を base environment と誤表示	server-resolved import manifest、direct/transitive closure check
Callback replay / confusion	二重 credit、別 submission への結果付替え	HMAC、nonce / idempotency、submission + job + digest binding
Account takeover	private ledger / identity 侵害	passkey / MFA、secure cookie、reset hardening、session revocation、audit log
Secret disclosure	infrastructure takeover	managed secret store、short-lived token、rotation、redaction runbook
Database compromise	private data exposure / ledger mutation	least privilege、encrypted backup、PII separation、external transparency root
Supply-chain compromise	judge image / action / dependency 改変	digest pin、signed build provenance、dependency review、two-person release approval

8.2 Fail-closed resource enforcement

Public sample logs observed on 2026-08-07 contain a warning that memory-limit capability was unavailable, while the job completed successfully [11]. This observation does not by itself establish exploitability or root cause, but it is sufficient to define a P0 control: production workers must refuse jobs when declared resource limits are not actually enforced.

Production invariant

`LimitsEnforced = false` は warning ではなく infrastructure failure とする。Proof failure、time-out、infrastructure failure、policy rejection を別 status にし、user に修正可能性を正確に伝える。

8.3 Secret handling and incident response

Credential、VPN profile、invite link、service token を chat、Issue、commit、shared note に貼らない。secret manager と short-lived credential を使用する。漏えいが疑われる場合は、削除より先に失効、scope 確認、access-log review、再発行、incident timeline、再発防止を行う。operator account には MFA / passkey を必須とする。

9 Public beta snapshot and trust reset

9.1 Snapshot: 2026-08-07

公開 API と repository から確認した current snapshot は次の通りである [9, 10, 17]。

Indicator	Observed value	Interpretation
Accepted certificates	387	supply は既に存在
Distinct external accepted provers	2	external adoption は初期段階
Reviewed certificates	0	semantic review network 未成立
Curated certificates	0	curation 実績未成立
No-disallowed-axiom rate	95.1% (368/387)	policy metric は存在、category audit が必要
Active pinned environment	Lean / Mathlib v4.13.0	historical reproducibility を保持すべき
Upstream stable Lean	v4.32.2	active と latest を分離表示すべき
Open Issues / PRs in public judge repo	0 / 0	public feedback / roadmap 導線が弱い
Public pricing	beta、paid disabled	monetization より reliability gate が先

9.2 Observed trust gaps

公開 sample submission では、UI が再計算した generated file hash と stored hash の不一致が表示される。また、その certificate では environment epoch や lake manifest hash 等の provenance field が未記録の例がある [11, 12]。これは「proof が誤り」と即断する根拠ではないが、「表示された source が exact judge input であり、第三者が完全に再現できる」という L2 claim を弱める。

さらに active pinned environment v4.13.0 に対し、公開 metric label は *Reverified on latest Lean* と表現する。2026-08-07 時点の upstream stable は v4.32.2 であるため、*Reverified on active TheoremForces environment* と *Upstream version lag* に分ける方が正確である [5, 17]。

9.3 Trust reset plan

1. 新規機能と paid launch を一時 freeze。
2. 全 387 certificate の artifact、dependency、environment、batch provenance を機械監査。
3. complete / legacy-incomplete / artifact-mismatch / dependency-mismatch / reverification-failed に分類。
4. mismatch / incomplete を fully reproducible と表示しない。
5. exact generated source を immutable blob として保存・配信。
6. schema v2 と verifier CLI を導入。
7. sandbox limit preflight を fail closed に変更。
8. v4.32.2 への shadow migration を実施し、failure taxonomy を公開。
9. stats API を唯一の集計元にして page 間の counter / label を統一。
10. public repository の branding、architecture、security policy、Issue workflow を現行化。

Trust reset exit gate

新規 certificate の exact artifact hash、dependency record、sandbox enforcement が 100% 一致し、restore drill が成功し、public label inconsistency が 0、critical integrity incident なしで 14 日間運用できた時点で完了とする。

10 Research workflow and interoperability

10.1 Flagship collection

TheoremForces の価値は certificate 数ではなく、実研究の problem decomposition、proof reuse、citation に現れる。flagship collection は最低限、次を一つの graph として提示する。

- informal problem と paper / preprint / notebook の source
- formal target と intent mapping
- definition、auxiliary lemma、bridge lemma、target theorem の semantic role
- certificate dependency DAG
- exact reproducibility bundle
- AI assistance と human contribution disclosure
- known limitation、review status、migration status
- BibTeX / LaTeX / Markdown / JSON citation

- mathlib へ upstream すべき general-purpose component

初期候補は、公開 ledger に連鎖が見える Eisenstein integer / quadratic form 系列である。第二候補として、研究者が実際に抱える inverse problem、geometry、analysis 等を選び、単発の toy problem と研究上の target theorem を区別する。

10.2 Temporal claim consistency pilot

佃氏の提案する「政治家の過去と現在の発言の矛盾を探す platform」は、certificate layer の非数学分野への application pilot になり得る。ただし出力は人物の真偽スコアや自動 **lie detector** ではなく、出典付き発言を形式化したときに成立する限定的な **temporal claim inconsistency certificate** とする。

各発言 evidence object は、原文、speaker、発言日時、source URL と archive / content digest、役職 / jurisdiction、対象期間、量化範囲、条件節、modality（事実・予測・公約・評価）、翻訳、formal proposition、formalizer を持つ。二つの発言 a, b から直ちに矛盾を宣言せず、同じ正規化 scope σ において $F(a, \sigma) = P$ かつ $F(b, \sigma) = \neg P$ を人間が確認し、その formal target に対する Lean proof を一つの L1 attempt として検証する。

Layer	pilot での object	保証しないこと
L1	normalized propositions から contradiction を導く単一 theorem	引用の真正性、翻訳、文脈
L2	source snapshot、日時、speaker、digest、formalization provenance	formalization が唯一の解釈であること
L3	source correction、archive 更新、ontology migration 後の reverification	historical record の消去
L4	scope、時制、条件、役職、引用文脈の human review	人物への道徳的・法的評価
L5	公益性、教育・研究価値に基づく curated collection	popularity による真実性

政策変更、見解の撤回、予測と後日の事実、異なる期間 / jurisdiction / population、編集された引用は、単独では logical contradiction と扱わない。UI は **formal inconsistency under reviewed scope**、**context disputed**、**not comparable**、**corrected** を区別し、formal statement と source mapping を公開する。

Public-figure pilot の governance gate

政治的中立な採用規則、複数媒体への source diversity、原文 archive、二名以上の context review、利益相反開示、本人 / 関係者の right of reply、迅速な correction / appeal、defamation・privacy・

選挙期の法務確認が必要である。この gate を満たすまで人物別ランキング、truth score、自動拡散機能を実装しない。

初期実験は少数の公開記録に限定し、(1) 比較可能性判定、(2) formalization の再現性、(3) review disagreement、(4) correction latency を測る。これは core beta の trust reset を遅らせない独立 design-partner pilot とする。

10.3 Relationship to mathlib

Mathlib は Lean 4 の shared mathematical library であり、source、cache、dependency workflow を公開している [18]。TheoremForces は次の bridge を提供する。

1. exploratory certificate から reusable lemma を特定する。
2. dependency と axiom status を整理する。
3. naming、generality、documentation、tests を mathlib contribution standard に合わせる。
4. TheoremForces certificate を approval として扱わず、通常の mathlib review process へ提出する。
5. upstream merge 後は、certificate の dependency を canonical mathlib theorem へ migration できるようにする。

10.4 Academic citation

Citation は platform homepage ではなく immutable certificate version を指す。推奨 citation bundle は title、authors / attribution、certificate id、certificate hash、formal statement hash、environment epoch、access date、permanent URL、related paper DOI を含む。public attribution は privacy request により mutable であり得るため、certificate hash から個人情報を分離する。

10.5 Preservation tiers

- **Online:** site、API、certificate JSON、source artifact。
- **Exportable:** self-contained tar / OCI bundle、manifest、verification script。
- **Replicated:** operator と異なる provider / Git repository に batch root と public artifacts を mirror。
- **Migrated:** historical epoch と active epoch の両方で status を追跡。

Reproducible Builds project が示すように、同一 source だけでなく、tool、environment、instructions を記録し、第三者が再実行して artifact を比較できる経路が必要である [19]。

11 Governance and sustainable operation

11.1 Small-team operating model

TheoremForces は public documentation 上、single independent maintainer の hobby project であり SLA を持たない [2]。この制約を隠さず、同時進行 epic を最大 2、weekly triage を 30 分、monthly

roadmap review を 1 回とする。

Area	Primary responsibility	Required second check
Research vision / flagship curation	mathematics lead	infrastructure feasibility
Judge / sandbox / observability	infrastructure lead	trust-policy review
Certificate schema / public claims	joint	two-person approval
Security incident	infrastructure lead	timeline and disclosure review
Academic outreach / case study	mathematics lead	participant consent
Paid pilot / legal entity	joint	gate review and external advice

11.2 Change governance

Trust-critical change は Issue、design note、tests、review、rollback plan、changelog を必要とする。特に次は two-person approval とする。

- acceptance predicate、axiom policy、wrapper template
- certificate canonicalization / hash algorithm
- environment epoch switch
- judge image、sandbox policy、secret scope
- certificate invalidation / retraction
- pricing と reputation policy

11.3 Business boundary

Public judge、public certificate、public ledger は free のまま維持する。paid feature は private ledger、collaboration、export、migration、citation tooling といった hosted software に限定し、Accepted、Seeds、ranking、review outcome、judge priority を販売しない [8]。

Paid pilot は信頼性、継続利用、support burden、data export、cost accounting の gate を満たした場合のみ開始する。初期は Personal と Team の二 tier に絞り、需要が確認される前に Pro / Pro+ / Enterprise を細分化しない。法人化は象徴的 milestone ではなく、継続売上、組織契約、liability separation、contributor payment の必要が生じた時点で判断する。

12 Roadmap: 2026-08 to 2027-07

Period	Theme	Deliverables	Exit gate
2026-08	Trustworthy beta	credential rotation、全 certificate audit、single-target L1、exact artifact storage、schema v2、certificate registry、sandbox fail-closed、restore drill	14 days with no critical integrity incident
2026-09 to 10	Research lighthouse	batch facade、flagship collection 2、design partner 5-8、v4.32.2 shadow migration、verifier alpha	external prover 5、certified dependency reuse 1
2026-11 to 2027-01	Review and retention	review dimensions、dispute / retraction、curated collections、temporal consistency pilot design、verifier stable	external prover 10、reviewed 5、curated 2
2027-02 to 04	Paid pilot decision	private ledger design partner、cost model、export / deletion / downgrade、support measurement、two-tier offer	2-3 paid pilots or explicit no-launch decision
2027-05 to 07	Evidence-driven scale	redundant judge、quarterly migration、independent verifier、transparency report、research collections 3-5	external prover 25、reviewed 20、curated 10

12.1 Phase 0: first 48 hours

1. suspected exposed credentials を失効・再発行し、operator MFA を有効化。
2. misleading *latest* / *fully reproducible* label を修正。
3. certificate minting に integrity kill switch を追加。
4. public sample mismatch を incident / integrity Issue として記録。

12.2 Phase 1: trustworthy beta

最初の一か月は feature freeze とする。全 387 certificate を audit し、schema v2 と verifier を先に作る。toolchain migration は historical record を破壊せず、新 epoch で shadow build してから default を切り替える。public repository には SECURITY.md、architecture、reproduction instructions、Issue templates、Milestones を追加する。同時に one attempt = one theorem の validator、primitive axiom

と certified dependency の分離、same-environment registry checkpoint を導入する。certificate の追加は registry root の追記とし、primitive policy version は変更しない。

12.3 Phase 2: research lighthouse

5-8 人の invite-only design partner と一緒に onboarding し、sign-up から first accepted certificate までを観察する。median time-to-first-certificate 30 分以内を目標とし、上位 5 blocker を修正する。research case study は「短時間で解けた」という marketing claim だけでなく、problem decomposition、human / AI contribution、failed attempts、dependency reuse、paper connection を記録する。

12.4 Phase 3: review and retention

Review quorum を実際に成立させ、外部 user が 4 週間後も戻る理由を作る。leaderboard より、collection progress、dependency unblock、migration health、review request を notification の中心にする。monthly research session では、未解決 problem 一つと certificate chain 一つに集中する。

12.5 Phase 4: paid pilot decision

Paid launch は deadline ではなく decision gate である。private ledger を 4 週間以上継続利用する design partner が 3 組以上、support が週 3 時間以内、cost / judge run と cost / active ledger が説明可能、90 日間 trust gate 維持、export / deletion / incident notification が動作する場合にだけ pilot を開く。

12.6 Phase 5: evidence-driven scale

Scale は user count ではなく verified demand に合わせる。judge redundancy、queue failover、quarterly migration、annual transparency report、independent verifier implementation を導入する。community との関係は mathlib の代替ではなく、reproducible staging / citation layer として築く。

13 Metrics and decision gates

13.1 North Star

$$\text{North Star} = \# \left\{ \begin{array}{l} \text{provenance-complete certificates authored by external users} \\ \text{and reused by another external proof or cited research output} \end{array} \right\}. \quad (8)$$

この metric は supply、external adoption、reuse、research relevance、provenance completeness を同時に要求する。

13.2 Dashboard

KPI	2026-08 gate	2027-01	2027-07
New certificate artifact hash match	100%	100%	100%
Dependency record match	100%	100%	100%
Sandbox limit enforcement	100%	100%	100%
Backup restore drill	pass	quarterly pass	quarterly pass
Distinct external accepted provers	baseline 2	10	25
Monthly retained external provers	instrument	4	10
Reviewed certificates	baseline 0	5	20
Curated certificates	baseline 0	2	10
Externally reused certificates	instrument	3	10
Public research case studies	0	2	3-5
Paper / preprint citations	instrument	1	3

Operational dashboard には p50 / p95 queue wait、judge runtime、timeout rate、infrastructure failure rate、proof failure rate、cost / run、active private ledger storage cost、weekly support time、security / integrity incident count を含める。

13.3 Feature decision rule

新機能は次の順で判定する。

1. certificate trust を上げるか。
2. real research の reuse / citation を増やすか。
3. external user の first value / retention を改善するか。
4. small part-time team で継続運営できるか。
5. measurable hypothesis があるか。

一つも満たさない場合は作らない。

14 Research questions outside the L1 acceptance meaning

次の問いは product / governance research であり、CertificateCore、canonicalization、signature、checker quorum、artifact identity、retention、log consistency、Gate 0-C を変更または延期する理由にはならない。

1. **Independent witness selection:** 必須の二 witness を超えて、どの operator / jurisdiction に mirror すれば split view 検出を強められるか。
2. **Batch semantics:** 複数定理 UI の部分成功、retry、collection manifest をどう表示し、atomic acceptance と誤認させないか。
3. **Registry checkpoint UX:** retained event prefix と current state map の差を、利用者にどう簡潔に説明するか。
4. **Certificate invalidation:** dependency の失効時に downstream taint と再検証をどの順序で伝播するか。
5. **Environment policy:** monthly Lean release cadence に対し、何 release 以内を target にするか。
6. **Review quorum:** 5-person quorum を維持しつつ、初期 reviewer shortage をどう解消するか。
7. **Privacy UX:** tenant encryption と cryptographic erasure の保証範囲を、private user にどう示すか。
8. **Persistent identifiers:** project-specific certificate id に DOI / SWHID 等を接続するか。
9. **AI benchmarking:** generated proof の model / prompt provenance を privacy と reproducibility の範囲でどこまで記録するか。
10. **Upstream bridge:** mathlib merge 後の certificate dependency migration を自動化するか。
11. **Temporal claims:** statement ontology、比較可能性、appeal、right of reply を政治的に中立な rule としてどう監査するか。

15 Conclusion

Formal mathematics が人間だけでなく AI system によって大量に生成されるとき、価値の中心は「proof がある」という claim から、何が検査され、どの artifact と environment が使われ、どの assumptions と dependencies を持ち、誰が意味を review し、誰が再現したかという provenance graph へ移る。

TheoremForces は、その graph を公開 certificate layer として構築する機会を持つ。すでに judge、ledger、axiom report、private workspace、semantic review model の基礎がある。一方で、external adoption と review は初期であり、public artifact と provenance claim の一致には trust-critical な beta debt が残る。

したがって次の一年の勝ち筋は明確である。まず exactness を修復し、次に real research で価値を示し、その後に少人数の external community と review network を育てる。課金と scale は、その証拠の後にのみ進める。この順序を守ることで、TheoremForces は proving game ではなく、human-AI formal mathematics のための長期的な public verification infrastructure になり得る。

付録 A Certificate v2 envelope example

次は三層 topology の縮約例である。掲載した field names とその object topology は normative JSON Schema に一致するが、紙幅のため required field の一部を省略している。この紙面例を test vector にせず、repository の完全 fixture を schema validation と golden-vector test に使う。Core に signature

と receipt は入らない。

Listing 3: Cycle-free certificate envelope topology

```
{
  "schema": "theoremforces.l1-certificate/2",
  "core": {
    "schema": "theoremforces.l1-core/2",
    "certificate_id": "tf-l1-01JEXAMPLE",
    "attempt_id": "01JATTEMPT",
    "issued_at": "2026-08-09T12:00:00Z",
    "event_sequence": 42,
    "artifact": {
      "upload_raw_sha256": "sha256:...",
      "upload_raw_uri": "cas://sha256/...",
      "entrypoint_raw_sha256": "sha256:...",
      "entrypoint_raw_uri": "cas://sha256/...",
      "manifest_sha256": "sha256:...",
      "manifest_uri": "cas://sha256/...",
      "bundle_root_sha256": "sha256:...",
      "generated_source_sha256": "sha256:...",
      "generated_source_uri": "cas://sha256/...",
      "proof_export_sha256": "sha256:...",
      "proof_export_uri": "cas://sha256/..."
    },
    "target": {
      "challenge_id": "tf-challenge-01JEXAMPLE",
      "challenge_artifact_sha256": "sha256:...",
      "challenge_artifact_uri": "cas://sha256/...",
      "user_declaration_count": 1,
      "fqname": "TheoremForces.Attempt.solution_target",
      "declaration_kind": "theorem",
      "type_sha256": "sha256:...",
      "value_sha256": "sha256:...",
      "proof_export_sha256": "sha256:..."
    },
    "environment": {
      "logical_digest": "sha256:...",
      "manifest_sha256": "sha256:...",
      "image_digest": "sha256:...",
      "architecture": "linux/amd64"
    },
    "certified_dependencies": {
      "status": "pass",
      "registry_checkpoint": "sha256:...",
      "items": [],
      "closure_sha256": "sha256:..."
    },
    "axioms": {
      "status": "pass",
      "policy_sha256": "sha256:...",
      "primitive_items": [],
      "closure_sha256": "sha256:..."
    },
    "comparator": {"status": "pass", "evidence_sha256": "sha256:..."},
    "checkers": {"status": "pass", "items": [
      {"role": "lean_checker", "status": "pass"},
      {"role": "independent_checker", "status": "pass"}
    ]},
    "run": {"outcome": "completed", "structured_result_count": 1},
    "l1": {"status": "accepted", "reason": "all_predicates_passed"}
  },
  "issuance_attestation": {
    "schema": "theoremforces.l1-issuance-envelope/1",
    "payload": {
      "schema": "theoremforces.l1-issuance/1",
      "core_sha256": "sha256:...",
      "job_expectation_sha256": "sha256:...",
      "result_attestation_sha256": "sha256:...",
      "issuer": {"key_id": "tf-issuer-2026-01", "algorithm": "Ed25519"},
      "issued_at": "2026-08-09T12:00:00Z"
    },
    "signature": "..."
  },
  "log_receipt": {
    "schema": "theoremforces.l1-log-receipt/2",
    "core_sha256": "sha256:...",
    "issuance_sha256": "sha256:...",
    "leaf_hash": "sha256:..."
  }
}
```

```

"tree_size": 42,
"leaf_index": 41,
"inclusion_path": [],
"checkpoint": {
  "schema": "theoremforces.l1-log-checkpoint-envelope/2",
  "payload": {
    "schema": "theoremforces.l1-log-checkpoint/2",
    "log_id": "tf-log-v2",
    "algorithm": "rfc6962-sha256",
    "tree_size": 42,
    "root_sha256": "sha256:...",
    "maximum_merge_delay_ms": 300000
  },
  "signature": "..."
},
"consistency": {
  "old_tree_size": 41,
  "old_root_sha256": "sha256:...",
  "consistency_path": []
},
"witness_receipts": [
  {
    "schema": "theoremforces.l1-witness-envelope/2",
    "payload": {
      "witness": {
        "witness_id": "witness-a"
      },
      "signature": "..."
    }
  },
  {
    "schema": "theoremforces.l1-witness-envelope/2",
    "payload": {
      "witness": {
        "witness_id": "witness-b"
      },
      "signature": "..."
    }
  }
]
}

```

付録 B Verification algorithm

Listing 4: Reference verification pseudocode

```

function verify(certificate_id, mode):
  e = parseStrictJsonAndValidateSchema(downloadEnvelope(certificate_id))
  core_sha256 = domainSeparatedJcsDigest(e.core)
  issuance_sha256 = issuanceDigest(e.issuance_attestation.payload)
  require e.issuance_attestation.payload.core_sha256 == core_sha256
  require verifyEd25519Envelope(e.issuance_attestation)
  require issuerKeyValidAtCheckpoint(e.issuance_attestation)

  blobs = downloadCasObjects(e.core.artifact)
  require allExactHashesMatch(blobs, e.core.artifact)
  require canonicalManifestAndBundleRootMatch(blobs)
  require jobExpectationMatchesResultAttestation(e.core)
  source = blobs.generated_source

  direct = resolveDependencies(e.core.certified_dependencies.items)
  require transitiveClosureHash(direct) ==
    e.core.certified_dependencies.closure_sha256
  require certificateGraphIsAcyclic(direct)
  require everyCertificateValidAtIssueCheckpoint(direct)
  require showCurrentTaintSeparately(direct)
  require fullChainCheckerReceiptsPass(direct)

  require verifyLogLeaf(core_sha256, issuance_sha256, e.log_receipt)
  require verifyInclusionAndConsistency(e.log_receipt)
  require verifySignedCheckpointAndTwoWitnesses(e.log_receipt)

  if mode in {REBUILD, MIGRATION}:
    require explicitSandboxConsent(mode)
    env = materializeEnvironment(e.core.environment, mode)
    result = sandboxedLakeBuild(env, source, direct)
    require comparatorPass(result)
    require exportedProofDigestMatches(result, e.core.target)
    require leanCheckerPass(result) and independentCheckerPass(result)
    require structuredAxiomClosureMatches(result, e.core.axioms)
    require exactResourceOutcomeMatches(result, e.core.run)

  return assuranceVector(e, currentRegistryState())

```

付録 C Operational SLO proposal

Public beta は formal SLA を約束しない。ただし内部 SLO を持ち、capacity と paid readiness の判断に使う。

Signal	Initial objective	Response
Certificate integrity	100%	mismatch 1 件で mint pause と audit
Sandbox preflight	100%	failure worker を pool から除外
Queue availability	99.0% monthly	status update、capacity review
p95 queue wait	under 5 min	concurrency / fairness tuning
p95 judge runtime	under 10 min	timeout taxonomy と challenge-specific review
Infrastructure error rate	under 2%	automatic retry once、root-cause bucket
Trust-critical RPO	0	CAS mirror + witnessed log; any loss pauses mint
Rebuildable analytics RPO	24 h	daily verified export
Restore drill	monthly	restored head must match independent witnesses
Critical incident page	within 5 min	public status within 60 min; user notice within 24 h

付録 D Glossary (用語集)

Accepted / Lean acceptance

Recorded environment で acceptance predicate を満たした submission。Lean による機械的受理であり、数学的 endorsement ではない。

Active pinned environment

TheoremForces が新規判定や再検査の標準として現在固定している Lean、Mathlib、manifest、judge image、policy の組。上流の最新安定版とは一致しない場合がある。

Artifact

Judge が実際に処理した generated Lean source と bundle contents。

Certificate

Artifact、environment、axiom、dependency、result を content-addressed metadata として固定した record。

Conjecture

User が証明対象として提示する formal target と human-readable intent。

Endorsement

Lean の型検査成功とは別に、人間または community が formal statement と意図の一致や数学的価値を認めること。本書では主に L4 (Intent review) と L5 (Curation) に対応する。

Environment epoch

Lean、Mathlib、manifest、judge image、policy をまとめた versioned execution environment。

Intent review

Formal statement と informal mathematical claim の alignment review。

Migration

Historical certificate を別 environment epoch で再検査し compatibility status を記録すること。

Provenance complete

Exact artifact と生成・実行・依存 metadata が第三者検算可能な状態。

Reverified

Original acceptance とは別に、指定 environment で再度 build に成功した状態。

Semantic provenance

Formal statement がどの informal intent、定義、依存関係、review record と結びつき、どう解釈・評価されたかを追跡できる意味上の来歴。Kernel acceptance や artifact provenance とは分けて記録する。

Seeds

Platform 内 reputation point。money、token、redeemable asset ではない。

Upstream stable release

Lean upstream project が公開している最新の正式安定版。TheoremForces が active pinned environment へ採用済みであることは意味しない。

References

参考文献

- [1] TheoremForces Project, "The public certificate layer for reproducible Lean proofs," <https://theoremforces.org/>, accessed 2026-08-07.
- [2] TheoremForces Project, "About TheoremForces," <https://theoremforces.org/docs/about>, accessed 2026-08-07.
- [3] TheoremForces Project, "How TheoremForces judges a proof," <https://theoremforces.org/docs/judging>, accessed 2026-08-07.
- [4] TheoremForces Project, "Security model," <https://theoremforces.org/docs/security>, accessed 2026-08-07.
- [5] TheoremForces Project, "Trust metrics - public ledger reliability," <https://theoremforces.org/docs/metrics>, accessed 2026-08-07.
- [6] TheoremForces Project, "Theorem Certificate Ledger," <https://theoremforces.org/docs/ledger>, accessed 2026-08-07.
- [7] TheoremForces Project, "Semantic provenance," <https://theoremforces.org/docs/semantic-pr>

- ovenance, accessed 2026-08-07.
- [8] TheoremForces Project, "Pricing," <https://theoremforces.org/pricing>, accessed 2026-08-07.
 - [9] TheoremForces Project, "Public stats API," <https://theoremforces.org/api/public/stats>, snapshot retrieved 2026-08-07.
 - [10] TheoremForces Project, "theoremforces-judge," GitHub repository, <https://github.com/ryendo/theoremforces-judge>, accessed 2026-08-07.
 - [11] TheoremForces Project, "Submission cmq1kfbwt000xs41rafzdl4iz," <https://theoremforces.org/submissions/cmq1kfbwt000xs41rafzdl4iz>, accessed 2026-08-07.
 - [12] TheoremForces Project, "Certificate tcl-cert-20260605-0d2nux," <https://theoremforces.org/ledger/certificates/tcl-cert-20260605-0d2nux>, accessed 2026-08-07.
 - [13] Lean Project, "The Lean Language Reference," <https://lean-lang.org/doc/reference/latest/>, accessed 2026-08-07.
 - [14] Lean Project, "Elaboration and Compilation: The Kernel," <https://lean-lang.org/doc/reference/latest/Elaboration-and-Compilation/>, accessed 2026-08-07.
 - [15] Lean Project, "Axioms," <https://lean-lang.org/doc/reference/latest/Axioms/>, accessed 2026-08-07.
 - [16] Lean Project, "Validating a Lean Proof," <https://lean-lang.org/doc/reference/latest/ValidatingProofs/>, accessed 2026-08-07.
 - [17] Lean Project, "Lean 4 releases," <https://github.com/leanprover/lean4/releases>, accessed 2026-08-07.
 - [18] Lean Community, "mathlib4: The math library of Lean 4," <https://github.com/leanprover-community/mathlib4>, accessed 2026-08-07.
 - [19] Reproducible Builds Project, "Reproducible Builds," <https://reproducible-builds.org/>, accessed 2026-08-07.
 - [20] OpenSSF, "SLSA Specification v1.2: Provenance," <https://slsa.dev/spec/v1.2/provenance>, accessed 2026-08-07.
 - [21] in-toto Project, "in-toto Attestation Framework," <https://github.com/in-toto/attestation>, accessed 2026-08-07.
 - [22] B. Laurie et al., "Certificate Transparency Version 2.0," RFC 9162, IETF, 2021, <https://www.rfc-editor.org/rfc/rfc9162.html>.
 - [23] A. Rundgren et al., "JSON Canonicalization Scheme (JCS)," RFC 8785, IETF, 2020, <https://www.rfc-editor.org/rfc/rfc8785.html>.
 - [24] S. Josefsson and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," RFC 8032, IETF, 2017, <https://www.rfc-editor.org/rfc/rfc8032.html>.
 - [25] Lean Project, "Comparator: validate untrusted Lean proofs against trusted challenges," <https://github.com/leanprover/comparator>, accessed 2026-08-09.